



Note

Longest increasing subsequences in sliding windows

Michael H. Albert^a, Alexander Golynski^b, Angèle M. Hamel^{c,*},
Alejandro López-Ortiz^b, S. Srinivasa Rao^b, Mohammad Ali Safari^b

^aDepartment of Computer Science, University of Otago, Dunedin, New Zealand

^bSchool of Computer Science, University of Waterloo, Waterloo, Ont., Canada N2L 3G1

^cDepartment of Physics and Computer Science, Wilfrid Laurier University, Waterloo, Ont., Canada
N2L 3C5

Received 12 June 2003; received in revised form 9 March 2004; accepted 17 March 2004

Communicated by A. Apostolico

Abstract

We consider the problem of finding the longest increasing subsequence in a sliding window over a given sequence (LISW). We propose an output-sensitive data structure that solves this problem in time $O(n \log \log n + \text{OUTPUT})$ for a sequence of n elements. This data structure substantially improves over the naïve generalization of the longest increasing subsequence algorithm and in fact produces an output-sensitive optimal solution.

© 2004 Elsevier B.V. All rights reserved.

Keywords: Longest increasing subsequence; Sliding window; Robinson–Schensted–Knuth

1. Introduction

Given a sequence $a_1 a_2 \dots a_n$ of distinct values from a linearly ordered set its *longest increasing subsequence* (LIS) is a subsequence of maximum length, whose values increase as the indices increase. The underlying set of the given sequence can be, and usually is, taken to be $\{1, 2, \dots, n\}$, so that the sequence can be viewed as a permutation $\pi = \pi(1)\pi(2) \dots \pi(n)$. In this setting the LIS consists of a sequence of indices

* Corresponding author. Tel.: +1-5198840710; fax: +1-5197460677.

E-mail addresses: malbert@cs.otago.ac.nz (M.H. Albert), agolynski@uwaterloo.ca (A. Golynski), ahamel@wlu.ca (A.M. Hamel), alopez-o@uwaterloo.ca (A. López-Ortiz), ssrao@uwaterloo.ca (S. S. Rao), masafari@uwaterloo.ca (M. Safari).

$1 \leq i_1 < i_2 < \dots < i_k \leq n$ such that $\pi(i_1) < \pi(i_2) < \dots < \pi(i_k)$ where k is the largest number for which such a sequence exists.

The *longest increasing subsequence problem* refers either to identifying the longest increasing subsequence(s) or, alternatively, to determining the length k of the LIS. In either of these forms, this problem has been the subject of intense study by mathematicians and computer scientists alike (see Section 2.1 for a more detailed discussion of previous work). This problem has interesting properties both from a purely combinatorial perspective (see e.g. [13]) as well as actual applications in fields such as DNA sequence matching [6]. This problem should not be confused with the longest common subsequence (LCS) problem which considers two sequences and locates a series of entries that appear in the same order in both sequences. However, LIS is a subcase of LCS.

In this paper, we consider the problem of finding the length of a longest increasing subsequence in every window of a sequence (LISW) of given width w ; that is, the LISs of substrings of π of the form $\pi(i+1)\pi(i+2)\dots\pi(i+w)$. This work is inspired by the study of the relationship and theoretical underpinnings of a problem and its windowed version [5,7]. We propose an output sensitive data structure which solves LISW in optimal time; that is, linear on the size of the output.

2. Problem definition

Given a sequence $\pi = \pi(1)\pi(2)\dots\pi(n)$ and a window size $w \leq n$, a window of π of width w is a subsequence $\pi(i+1)\pi(i+2)\dots\pi(i+w)$ for some $0 \leq i \leq n-w$. We also consider the **truncated windows** $\pi(1)\dots\pi(j)$ for $j \leq w$ and $\pi(j)\dots\pi(n)$ for $j \geq n-w$ as windows of size w . The general problem that we consider is that of determining a LIS in each of the windows W_i . Within this framework, several related questions can be posed regarding this problem, each with potentially different time complexity.

Local Max Value For each window report the length k of the longest increasing subsequence in that window.

Local Max Sequence Explicitly list a longest increasing sequence for each window.

Global Max Sequence Find the window with the longest increasing sequence among all windows.

We will deal with the Local Max Sequence form of the LISW. The algorithm we present runs in linear time on the size of the output for this problem and hence is optimal both in the worst case and adaptive sense. The same algorithm solves the other two versions of the problem described above, although its optimality in these cases is an open question.

2.1. Previous work

Algorithms for finding the length of the LIS date back to Robinson [12] and Schensted [14] with a generalization due to Knuth [10]. These algorithms have time complexity $O(n \log n)$ which is optimal in the comparison model. Hunt and Szmanski [9] give an algorithm with time complexity $O(n \log \log n)$ using the van Emde Boas data

structure [15]. Chang and Wang [4] also give an $O(n \log \log n)$ algorithm based on a permutation graph interpretation. Bespamyatnikh and Segal [3] present an $O(n \log \log n)$ algorithm that determines *all* longest increasing subsequences. The algorithm we present here is $O(n \log \log n + \text{OUTPUT})$. Probabilistic results related to this problem have been discussed in Aldous and Diaconis [1] and Groenboom [8]. The question also has application in bioinformatics in the MUMmer system for finding matches between DNA sequences [6].

Apostolico et al. [2] consider the problem of finding maximum cliques in circle graphs. This problem can be translated to a longest increasing subsequence problem. The approach in [2] is to restrict consideration to just part of the sequence at a time and search for an LIS in just that part of the sequence. However, the concept of window defined there is different than ours. One edge of their window is always one of the ends of the original sequence, so that their window is not a sliding window but is rather an expanding input sequence. Furthermore their window does not include all sequence elements so that, depending on the location of the edges in the circle graph, a window of size m could contain as many as m elements in the permutation or as few as zero.

2.2. Summary of contributions

LISW has an obvious naïve algorithm for the local max sequence which simply computes the LIS in each window separately. Using the methods described in the preceding section, this gives an algorithm whose complexity is $O(nw \log \log n)$. In the case where the average length of the LIS in each window is $\Theta(w)$ then, our algorithm offers no asymptotic improvement over this method.

However, it is well known in the permutation case that the average length of the LIS of a permutation of length n is asymptotically $2\sqrt{n}$ (see [1] for this result, and references). Suppose that a permutation π of length n is chosen uniformly at random. Consider any fixed window of π . The relative ordering of values observed in that window will also be uniformly chosen from among the patterns of permutations of length w . Thus the expected length of an LIS in any given window is asymptotically $2\sqrt{w}$ and by linearity of expectation, the expected total length of all LISs is $O(n\sqrt{w})$. So, in the expected case, or in any situation where the average length of the LIS in each window is $o(w)$, our algorithm offers a significant improvement on the naïve one. To wit, the naïve algorithm processes $(n - w + 1)$ windows and requires $O(w \log \log w)$ time for each, giving $O((n - w + 1)w \log \log w)$. For w sufficiently large, e.g. $w = cn$, given that the LIS in the whole sequence is length $O(\sqrt{n})$ our algorithm gives worst case bounds of $O(n^{1.5})$. Even for small w , given that the expected length of each LIS in each window is $O(\sqrt{w})$ then our algorithm gives $O(n\sqrt{w})$ (ignoring smaller order terms) while the naïve algorithm gives $O(nw \log \log w)$.

This improvement is obtained largely through the judicious use of a particular data structure. This data structure implicitly represents information pertinent to determining the LISs of the current window and to determining the LISs of all suffixes of the current window. This information can then be used to update the structure each time we drop an element off the beginning of the window and add one to the end. As with

3	35	25	257	247	2478	1478	1468	
		3	3	35	35	25	257	= P
						3	3	

Fig. 1. Tableau P created by Robinson–Schensted–Knuth Algorithm for $\alpha = 35274816$. First 3 is inserted in the tableau. Then 5 is inserted. As 5 is greater than 3 it is placed to the right of 3. Next 2 is inserted. As it is less than 3 it bumps 3 and takes its place while 3 goes to the second row. The insertion continues in this fashion.

most algorithms concerned with aspects of the LIS problem, our starting point will be the original constructions of Robinson and Schensted, so it will be helpful to review these next.

2.3. Tableaux and the Robinson–Schensted–Knuth algorithm

The Robinson–Schensted–Knuth algorithm (see Van Leeuwen [16] and references therein; for background see also Knuth [11] or Sagan [13]) is based on the concept of a tableau which can be used to determine increasing subsequences of a permutation. More formally,

Definition 2.1. A tableau of shape $\lambda = \lambda_1, \lambda_2, \dots, \lambda_m$ where $\lambda_1 + \lambda_2 + \dots + \lambda_m = n$ is a collection of n elements arranged in left-justified rows such that row i has λ_i elements, and the elements increase weakly across rows and increase strictly down columns. (See Fig. 1 for an example of a tableau.)

Although we concern ourselves mostly with permutations, we will discuss the Robinson–Schensted–Knuth algorithm in its full generality as applied to sequences of possibly repeated elements that come from a linearly ordered set. The algorithm we introduce in this paper uses a generalization of the Robinson–Schensted–Knuth algorithm and, in particular, uses the same “bumping” rules as Robinson–Schensted–Knuth.

Given a sequence $\alpha = \alpha_1, \alpha_2, \dots, \alpha_n$, the Robinson–Schensted–Knuth algorithm constructs a pair of tableaux P and Q both of shape λ where λ is a partition of n . We describe here just the construction of P as that includes the bumping technique we use. Given α , elements $\alpha_1, \alpha_2, \dots, \alpha_n$ are inserted one at a time in that order to form P . At step 1 place a single element, α_1 , as the first element of the first row of P . At step i , place α_i using the following algorithm: Scan the first row of P from left to right to locate the smallest element t that is greater than α_i . If no such element t exists, place α_i at the end of the first row of P . If t does exist, remove t from the first row of P and put α_i in its place. We say α_i *bumps* t . Then scan the second row of P from left to right to locate the smallest element in the second row of P that is greater than t . If no such element exists, place t at the end of the second row of P . If t does bump an element, insert that element into the third row of P and continue bumping elements until the currently bumped element comes to rest at the end of a row in P . Continue until all elements of α are exhausted (see Fig. 1 for an example).

The length of the first row of tableau P is equal to the length of the LIS for the permutation. This sequence can be determined via Schensted's basic subsequences. Schensted [14] defined the i th basic subsequence to be those elements that had occupied the i th position in the first row of P . It is easy to see that the basic subsequences are decreasing and that each element belongs to exactly one basic subsequence. Any longest increasing subsequence includes exactly one element from each basic subsequence and an increasing subsequence can be determined by associating each element a with the element b to its left when it entered the first row of P . This result shows the significance of the first row of the Robinson–Schensted–Knuth construction and indeed in our algorithm we make use of the first row only and discard the rest.

3. Algorithm

In order to deal with the problem of determining the longest increasing subsequences in the windows of a permutation we first consider a data structure which addresses a slightly more general question. In this structure we maintain information about the LIS of a sequence in such a way that we can:

- remove the first element of the sequence,
- add an element to the end of the sequence,
- query the data structure for the length of the current LIS.

For a given initial sequence $\alpha = \alpha_1 \alpha_2 \cdots \alpha_n$ let $\alpha_i^j = \alpha_i \alpha_{i+1} \cdots \alpha_j$ denote the subsequence from the i th to the j th element. We apply Robinson–Schensted–Knuth to α but keep track of only the first row in the tableau. We call this row the principal row of α and denote it by $P(\alpha)$. Our data structure will maintain principal rows for all the suffixes of the current sequence α ; that is, all the rows $P(\alpha_1^n), P(\alpha_2^n), \dots, P(\alpha_n^n)$. It will be helpful to think of these rows as lying one above the other in a *row tower* (see Fig. 2).

Now consider this data structure applied to the LISW problem, beginning with the subsequences $\{\pi(1)\}, \{\pi(1), \pi(2)\} \dots$ of a permutation π of length n .

The removal operation is easy: to remove the first element we need only delete the first row of the row tower. Adding a new element corresponds to inserting it using a Robinson–Schensted–Knuth approach in each of the rows stored so far and creating a new row consisting of this element only. The length of the LIS of the current window is the length of the first principal row we store.

Row Towers :

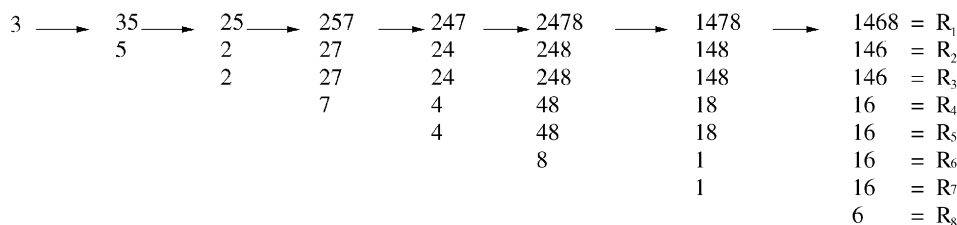


Fig. 2. Row towers for $\pi = 35274816$.

A naïve implementation of this data structure using a van Emde Boas priority queue [15] for each row takes $O(1)$ time for expiring, $O(w \log \log n)$ time for adding each element and $O(1)$ time for outputting the length of each subsequence. Total time complexity would be $O(nw \log \log n)$.

However, observe that each row other than the first in the row tower is either the same as the one above, or can be obtained from it by deleting a single element. This claim is easily verified by induction. In the trivial case this claim holds when the first element is added, since there is only a single row in this case. Now consider two consecutive rows before insertion of a new element b . If they are the same then they will remain the same after inserting b . Alternatively if they differ in a single element r , then if b does not bump r from the first row, they will still differ in the same way. If b does bump r , then either it bumps the next element of the second row, or is added to the end of that row. In the first case the two rows still differ by one deletion (the next element after r), while in the second case they are now the same. Thus we have proven:

Lemma 3.1. *Let sequence S be a suffix of sequence T . Then $P(S)$ is a subsequence of $P(T)$ and $|P(T)| - |P(S)| \leq |T| - |S|$.*

Since we now know that the row tower forms an inclusion chain we can remove duplicate rows and record the original multiplicity of remaining rows in a sequence m . From now on, when we refer to the row tower, we will assume that the rows have been made distinct in this way. After this modification the data structure still supports all the operations as described above, but the time complexity for adding is $O(\ell \log \log n)$ and space is $O(n\ell)$ at this time, where ℓ denotes the length of the current LIS.

Suppose that the first row of the row tower contains ℓ symbols. Then, to each position in this row, we associate the number of the last row in which this symbol occurs. Since each row differs from the preceding one by the removal of exactly one symbol, this gives a permutation σ on the elements $1, 2, \dots, \ell$. We call σ the drop out permutation of the row tower. We can also define a drop out sequence, d , of drop out times, by replacing each element of the drop out permutation by the actual index of the last row in which the corresponding element of the principal row occurs.

For the example in Fig. 2, we have $m = (1, 2, 4, 1)$, $d = (7, 3, 8, 1)$ and $\sigma = (3, 2, 4, 1)$. Fig. 3 illustrates the transformation of the row tower as the window slides.

We use the values of the principal row, d , and σ , as an implicit representation of all but the first row in the row tower. That is, this data structure has three components, the principal row $R = R_1$, the drop out sequence d , and the drop out permutation σ . Although it is clear that the first two of these suffice to describe the complete row tower, we will make use of the third when we wish to produce actual LISs from each window, rather than simply the length of the LIS in each window. Next we describe how to update these parts under expire and add operations.

The **expire operation** simply subtracts 1 from each element of d and deletes the element with expiry time 0 (if there is one) from R . If no deletion occurs then σ is unchanged. Otherwise, the element 1 is deleted from σ and the remaining values are decreased by 1.

Window size = 6

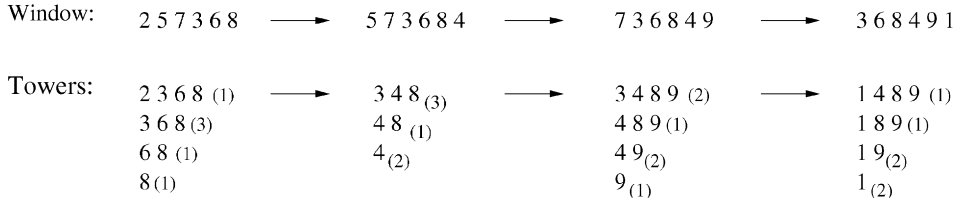


Fig. 3. Construction for $\pi = 257368491$.

The **add operation** for an element b requires that b be used to bump an element out of each row of the row tower (unless it is appended to all of them). Since, as we have observed, the rows form an inclusion chain, if b bumps a certain element s out of a row, then it bumps the element s out of all further rows to which s belongs. In other words, the drop out time for s changes to the index of the first row from which it is bumped by b . Now consider the next row of the row tower (if one exists) after s has dropped out. In this row there may or may not be elements larger than s . If there are such elements then b bumps the smallest of them. If not, then b is appended to the end of this and all subsequent rows.

So there is a sequence of indices $i_1 < i_2 < \dots < i_k$ for the sequence d defined as follows: i_1 is the least index of an element of the principal row which is larger than b (if no such index exists, then the sequence is empty), and i_{t+1} is the least index larger than i_t for which $d(i_{t+1}) > d(i_t)$. These indices represent the elements of the principal row which are bumped by b . Since b is placed in position i_1 in the first row and does not drop out until the very end, the sequence d is updated according to:

$$\begin{aligned} d(i_{t+1}) &= d(i_t) \quad \text{for } t = 1, 2, \dots, k-1, \\ d(i_1) &= w+1. \end{aligned}$$

Similarly, the update of σ is

$$\begin{aligned} \sigma(i_{t+1}) &= \sigma(i_t) \quad \text{for } t = 1, 2, \dots, k-1, \\ \sigma(i_1) &= \ell. \end{aligned}$$

At this point, we have a data structure with expire/add time $O(\ell)$ per element, query time for the length of the LIS in the current window $O(1)$ and space $O(n)$.

In order to support the operation of outputting an LIS we maintain a tree for each row R_i . In the tree associated to R_1 the paths from vertices to the root will constitute reversals of (some) increasing sequences in the current window. In particular, the path from the last element of R_1 to the root will be an LIS.

The reason for including multiple trees is to allow for the expiry operation. When the last element of a principal row expires, we consider the row itself to expire. At the point where a principal row expires, it will be necessary to have access to the tree for the new principal row. The basic idea is simply that whenever an element is added to

a row it is also added to the tree corresponding to that row and its parent in the tree is the element of the row immediately to its left. The property claimed of paths from vertices to the root then follows immediately.

However, the difficulty with this approach is that all but the first row have implicit representations, and hence looking up the predecessor of an element in each row as required above is a non-trivial operation. We overcome this difficulty by noting that each parent operation takes us one column to the left in some row tower. This row tower is not necessarily the current one, since the element whose parent we seek may already have been bumped from the current row tower, as happens for instance when we look for the LIS in 1342, the element 3 which is 4's parent, no longer occurs in the row tower after 2 arrives. Suppose that we have a value v and a column c that v occupies in some row tower. When v is first added to the row tower, there is a unique row in which it occupies column c . We set the parent of v in column c to be the predecessor of v in that row. In other words, at the time that v is added we establish an array whose entry in position c is the parent of v in column $c - 1$. This can easily be accomplished from the explicit information available.

Namely, when v is first added, it is added in, say, column C . Its predecessor in that column is its immediate predecessor, say p_1 , in the principal row. This remains its predecessor in columns $C - 1$ through $C - \sigma(p_1) + 1$. In column $C - \sigma(p_1)$ its parent will be the rightmost element p_2 of the principal row which satisfies $\sigma(p_2) > \sigma(p_1)$, and this will remain its parent through column $C - \sigma(p_2) + 1$. Thus, by scanning leftwards along the principal row we can create references to all the parents of v in each column. When we exhaust the elements to the left of v (that is, thinking in terms of the row tower, when we reach the final block of rows of which v is the initial element) the parent of v is simply set to the root element of the tree.

Now, we can construct the reversal of the LIS in a given window in constant time per element. Namely, we begin with the rightmost element of the principal row (column ℓ). Using the array associated with this element we determine its parent in column $\ell - 1$, the second (last) element of the LIS. In turn using the array associated with that element we find its parent in column $\ell - 2$, and so on.

Hence the data structure proposed computes longest subsequences on a sliding window, with a cost for the i th window of $O(\ell_i)$ where ℓ_i is the length of the longest increasing sequence in window i . Thus, the total time is given by $\sum_{i=0}^{n-w} \ell_i = \text{OUTPUT}$. However, in the beginning we need to initialize the data structure. Creating an empty van Emde Boas data structure can be done in $O(1)$ time. Adding a new element to the van Emde Boas data structure costs at most $O(\log \log n)$ time plus at most $O(1)$ for updating all additional structures described above. Thus the total initialization time is $O(w \log \log n + \sum_{i=1}^w \ell'_i) = O(w \log \log n + w^2)$, where ℓ'_i is the length of the LIS in $\pi(1), \pi(2), \dots, \pi(i)$.

Theorem 3.2. *The algorithm described above computes the n longest increasing sequences, one for each window, in total time $O(n \log \log n + \text{OUTPUT})$.*

As an interesting side benefit, the algorithm obtained computes the LISW in an on-line fashion.

4. Conclusions and open problems

We proposed a data structure for finding the longest increasing subsequence in a sliding window over a given sequence (LISW). The data structure uses an implicit representation of principal rows for each of the subsequences on a window, and results in an output-sensitive algorithm. This data structure substantially improves over the naïve generalization of the longest increasing subsequence algorithm. An on-line, output-sensitive optimal algorithm is derived from this data structure. The time complexity is $O(n \log \log n + \text{OUTPUT})$. In particular if we have $O(n)$ possible outputs of length $o(w)$ then our algorithm will always do better than the naïve algorithm.

Other variations of the problem remain open, in particular the exact time complexity of the global max sequence problem remains an open question. Another interesting case is the off-line case, in which a pre-processing step in $o(n \log \log n + \text{OUTPUT})$ time is allowed. Then a query is issued for the longest subsequence within a given window which must be answered in time $o(w \log \log n)$.

Acknowledgements

We wish to thank the participants of the Algorithmic Problem Session at the University of Waterloo for many helpful discussions.

References

- [1] D. Aldous, P. Diaconis, Longest increasing subsequences: from patience sorting to the Baik–Deift–Johansson theorem, *Bull. Am. Math. Soc.* 36 (1999) 413–432.
- [2] A. Apostolico, M.J. Atallah, S.E. Hambrusch, New clique and independent set algorithms for circle graphs, *Discrete Appl. Math.* 36 (1992) 1–24.
- [3] S. Bspamyatnikh, M. Segal, Enumerating longest increasing subsequences and patience sorting, *Inform. Process. Lett.* 76 (2000) 7–11.
- [4] M.-S. Chang, F.-H. Wang, Efficient algorithms for the maximum weight clique and maximum weight independent set problems on permutation graphs, *Inform. Process. Lett.* 43 (1992) 293–295.
- [5] M. Datar, A. Gionis, P. Indyk, R. Motwani, Maintaining Stream Statistics over Sliding Windows, *ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2002.
- [6] A.L. Delcher, S. Kasif, R.D. Feischmann, J. Peterson, O. White, S.L. Salzberg, Alignment of whole genomes, *Nucl. Acids Res.* 27 (1999) 2369–2376.
- [7] P.B. Gibbons, S. Tirthapura, Distributed Streams Algorithms for Sliding Windows, *Proceedings of the 14th ACM Symposium on Parallel Algs. and Architectures (SPAA)*, 2002, pp. 63–72.
- [8] P. Groeneboom, Hydrodynamical methods for analyzing longest increasing subsequences, *J. Comp. Appl. Math.* 142 (2002) 83–105.
- [9] J. Hunt, T. Szymanski, A fast algorithm for computing longest common subsequences, *Comm. ACM* 20 (1977) 350–353.
- [10] D.E. Knuth, Permutations, matrices, and generalized Young tableaux, *Pacific J. Math.* 34 (1970) 709–727.
- [11] D.E. Knuth, *The Art of Computer Programming*, Vol. 3, Sorting and Searching, Addison–Wesley, Reading, Mass, 1973.
- [12] G. de B. Robinson, On representations of the symmetric group, *Am. J. Math.* 60 (1938) 745–760.

- [13] B. Sagan, *The Symmetric Group*, Wadsworth and Brooks/Cole, Pacific Grove, Calif, 1991.
- [14] C. Schensted, Longest increasing and decreasing subsequences, *Can. J. Math.* 13 (1961) 179–191.
- [15] P. van Emde Boas, R. Kaas, E. Zijlstra, Design and implementation of an efficient priority queue, *Math. Systems Theory* 10 (2) (1976/77) 99–127.
- [16] M. Van Leeuwen, The Robinson–Schensted and Schützenberger algorithms, an elementary approach, *Electron. J. Comb. Foata Festschrift* 3 (2) (1996) R15.